Solving Linear Systems: Elimination, Iteration, and Energy

N-20260604

§1.1 One equation, three ways of thinking

The equation

$$Ax = b$$

looks like a single problem, but numerically it can mean three rather different things.

If the matrix is moderate in size and used several times, it is natural to *factor* it. If the matrix is huge and sparse, it may be better to improve a guess one step at a time. If A is symmetric positive definite, the equation is also the stationarity condition of a convex quadratic energy. These are not three unrelated collections of formulas. They are three views of the same linear map:

eliminate variables, propagate corrections, minimize energy.

A useful solver therefore starts by asking what structure is available. Dense or sparse? Symmetric or nonsymmetric? Positive definite or indefinite? One right-hand side or many? The answer often matters more than the nominal matrix size.

§1.2 Triangular systems are the basic currency

Nearly every direct method tries to reduce a general system to triangular ones. If L is lower triangular, then the first equation contains only x_1, the second contains x_1, x_2, and so on. Forward substitution gives

$$x_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j<i} l_{ij} x_j \right).$$

For an upper triangular matrix U, backward substitution runs in the opposite direction:

$$x_i = \frac{1}{u_{ii}} \left(b_i - \sum_{j>i} u_{ij} x_j \right).$$

The arithmetic cost is only $O(n^2)$. The expensive part of a direct solve is not substitution but producing the triangular factors. This explains why an LU or Cholesky factorization is especially valuable when many vectors b must be solved against the same matrix: the cubic work is paid once, while each new right-hand side needs only triangular solves.

§1.3 Gaussian elimination is a factorization in disguise

Consider

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{bmatrix}, \qquad b = \begin{bmatrix} 5 \\ -2 \\ 9 \end{bmatrix}.$$

The first pivot is 2. Replacing

$$R_2 \leftarrow R_2 - 2R_1, \qquad R_3 \leftarrow R_3 + R_1$$

gives

$$\left[\begin{array}{ccc|c} 2 & 1 & 1 & 5 \\ 0 & -8 & -2 & -12 \\ 0 & 8 & 3 & 14 \end{array}\right].$$

One more elimination step,

$$R_3 \leftarrow R_3 + R_2,$$

produces

$$U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 0 & 1 \end{bmatrix}.$$

Back substitution yields

$$x_3 = 2, \qquad x_2 = 1, \qquad x_1 = 1.$$

The multipliers used during elimination were $2, -1, -1$. Placed in a unit lower triangular matrix, they form

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{bmatrix}, \qquad A = LU.$$

Thus elimination is not merely a sequence of row tricks. It constructs a reusable representation of the linear map.

§1.4 Why pivoting is part of the algorithm

Exact arithmetic only objects to a zero pivot. Floating-point arithmetic also objects to a very small one. For example,

$$A = \begin{bmatrix} 10^{-12} & 1 \\ 1 & 1 \end{bmatrix}$$

is perfectly invertible, yet elimination without row exchange uses the multiplier 10^{12}. A tiny rounding error in the first row can then be magnified enormously.

Partial pivoting swaps the largest available entry in the current column into the pivot position. Algebraically the result is

$$PA = LU,$$

where P is a permutation matrix. Pivoting does not change the problem; it changes the order in which information is eliminated so that intermediate numbers remain under control.

This distinction is worth keeping: conditioning belongs to the original problem, whereas numerical stability belongs to the algorithm. A well-conditioned system can still be damaged by a poor elimination order, and a stable algorithm cannot make an intrinsically ill-conditioned system insensitive.

§1.5 Structure can make elimination cheaper

A general dense factorization costs $O(n^3)$, but special matrices often require much less.

If A is symmetric positive definite, Cholesky factorization writes

$$A = LL^T.$$

Only one triangular factor is needed, symmetry halves the storage, and the work is about half that of a general LU factorization. An LDL^T factorization separates scaling from elimination:

$$A = LDL^T,$$

with L unit lower triangular and D diagonal or block diagonal.

For a tridiagonal matrix,

$$A = \begin{bmatrix} d_1 & c_1 & & & \\ a_1 & d_2 & c_2 & & \\ & a_2 & d_3 & \ddots & \\ & & \ddots & \ddots & c_{n-1} \\ & & & a_{n-1} & d_n \end{bmatrix},$$

elimination preserves the band. The resulting Thomas algorithm needs only $O(n)$ work and storage. This is a first example of a general numerical principle: preserve sparsity whenever possible, because unnecessary fill-in can cost more than the original equations.

§1.6 When a sequence of guesses is better than a factorization

For a very large sparse matrix, even storing the factors may be too expensive. Iterative methods instead generate

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots$$

and hope that these vectors approach the solution.

Write

$$A = M - N.$$

Then

$$Mx = Nx + b,$$

so a natural iteration is

$$Mx^{(k+1)} = Nx^{(k)} + b.$$

Equivalently,

$$x^{(k+1)} = Bx^{(k)} + c, \qquad B = M^{-1}N, \quad c = M^{-1}b.$$

If x_* is the exact solution and $e^{(k)} = x^{(k)} - x_*$, then

$$e^{(k+1)} = Be^{(k)}, \qquad e^{(k)} = B^k e^{(0)}.$$

Therefore convergence for every initial guess is equivalent to

$$\rho(B) < 1.$$

This one equation explains why stationary iterations are spectral problems rather than merely recursive formulas.

§1.7 Jacobi and Gauss–Seidel seen on the same example

Let

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

The exact solution is

$$x_* = \begin{bmatrix} 1/10 \\ 3/5 \end{bmatrix}.$$

Starting from $x^{(0)} = (0, 0)^T$, Jacobi updates both coordinates using only the old iterate:

$$x_1^{(k+1)} = \frac{1 - x_2^{(k)}}{4}, \qquad x_2^{(k+1)} = \frac{2 - 2x_1^{(k)}}{3}.$$

The first two iterates are

$$x^{(1)} = \begin{bmatrix} 1/4 \\ 2/3 \end{bmatrix}, \qquad x^{(2)} = \begin{bmatrix} 1/12 \\ 1/2 \end{bmatrix}.$$

Gauss–Seidel immediately reuses the newest value of x_1:

$$x_1^{(k+1)} = \frac{1 - x_2^{(k)}}{4}, \qquad x_2^{(k+1)} = \frac{2 - 2x_1^{(k+1)}}{3}.$$

Thus

$$x^{(1)} = \begin{bmatrix} 1/4 \\ 1/2 \end{bmatrix}, \qquad x^{(2)} = \begin{bmatrix} 1/8 \\ 7/12 \end{bmatrix}.$$

Both converge here, but Gauss–Seidel moves more quickly because each sweep contains a small amount of implicit coupling.

Strict diagonal dominance,

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|,$$

is a convenient sufficient condition for convergence of both methods. It is not the only condition, and failure of diagonal dominance does not automatically imply divergence. The exact question remains $\rho(B) < 1$.

§1.8 Residuals are visible; errors are not

During an iteration the true error

$$e^{(k)} = x^{(k)} - x_*$$

is usually unknown, because knowing x_* would make the computation unnecessary. What can be computed is the residual

$$r^{(k)} = b - Ax^{(k)}.$$

Since

$$Ae^{(k)} = -r^{(k)},$$

we have

$$e^{(k)} = -A^{-1}r^{(k)}.$$

Hence a small residual implies a small error only after the inverse is controlled. In norm form,

$$\|e^{(k)}\| \le \|A^{-1}\| \, \|r^{(k)}\|.$$

For an ill-conditioned matrix, a visually tiny residual can still hide a noticeable solution error. Stopping criteria should therefore be interpreted together with scaling and condition estimates, not as universal certificates.

§1.9 Positive definite systems are convex optimization problems

Suppose $A = A^T$ is positive definite. Define

$$\phi(x) = \frac{1}{2} x^T A x - b^T x.$$

Then

$$\nabla \phi(x) = Ax - b, \qquad \nabla^2 \phi(x) = A \succ 0.$$

Thus ϕ is strictly convex, and its unique minimizer satisfies

$$Ax = b.$$

Completing the square gives an even clearer identity:

$$\phi(x) - \phi(x_*) = \frac{1}{2}(x - x_*)^T A (x - x_*) = \frac{1}{2}\|x - x_*\|_A^2,$$

where

$$\|v\|_A = \sqrt{v^T A v}.$$

Solving the linear system is therefore exactly the same as driving the A-energy error to zero.

Steepest descent follows the negative residual direction. With

$$r_k = b - A x_k,$$

choose

$$x_{k+1} = x_k + \alpha_k r_k.$$

Minimizing $\phi(x_k + \alpha r_k)$ along this line gives

$$\alpha_k = \frac{r_k^T r_k}{r_k^T A r_k}.$$

The method always decreases the energy, but for elongated level sets it tends to zigzag. The elongation is governed by the condition number $\kappa_2(A)$.

§1.10 Conjugate gradients remembers previous progress

Steepest descent repeatedly chooses the locally best direction, yet may undo part of earlier progress. Conjugate gradients avoids this by constructing search directions p_k that are A-orthogonal:

$$p_i^T A p_j = 0 \qquad (i \neq j).$$

Starting from x_0, set

$$r_0 = b - Ax_0, \qquad p_0 = r_0.$$

Then iterate

$$\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}, \qquad x_{k+1} = x_k + \alpha_k p_k,$$

$$r_{k+1} = r_k - \alpha_k A p_k, \qquad \beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k},$$

$$p_{k+1} = r_{k+1} + \beta_k p_k.$$

After k steps, x_k lies in the affine Krylov space

$$x_0 + \mathcal{K}_k(A, r_0), \qquad \mathcal{K}_k(A, r_0) = \operatorname{span}\{r_0, Ar_0, \dots, A^{k-1} r_0\},$$

and minimizes the energy over that space. In exact arithmetic, conjugate gradients reaches the exact solution in at most n steps. Its practical value is that useful accuracy often arrives much sooner, especially when the eigenvalues are clustered.

A standard bound is

$$\frac{\|x_k - x_*\|_A}{\|x_0 - x_*\|_A} \le 2 \left(\frac{\sqrt{\kappa_2(A)} - 1}{\sqrt{\kappa_2(A)} + 1} \right)^k.$$

This makes the role of conditioning explicit: poor geometry slows the polynomial approximation carried out by the Krylov method.

§1.11 Preconditioning changes the geometry, not the solution

A preconditioner M is a matrix that is easy to solve with and approximates A. Instead of attacking

$$Ax = b$$

directly, one solves an equivalent system such as

$$M^{-1} A x = M^{-1} b.$$

The solution is unchanged, but the spectrum may become much more favorable.

For symmetric positive definite problems, a symmetric preconditioner can be understood through

$$M^{-1/2} A M^{-1/2} y = M^{-1/2} b, \qquad x = M^{-1/2} y.$$

Good preconditioning clusters the eigenvalues of the transformed matrix and reduces its condition number. Diagonal scaling, incomplete Cholesky factors, and multigrid operators are increasingly sophisticated realizations of the same idea: solve an easier nearby problem to correct the hard one.

§1.12 Choosing a method without turning the chapter into a menu

The methods above are best remembered through the obstacle each one removes.

A dense system with several right-hand sides rewards a stable factorization such as pivoted LU. Positive definiteness makes Cholesky cheaper and supplies an energy interpretation. Tridiagonal or banded structure should be preserved rather than destroyed.

Very large sparse systems favor iteration because matrix–vector products are cheap while fill-in is expensive. Stationary methods are simple smoothers, but their speed is tied to one fixed iteration matrix. Krylov methods adapt a polynomial to the observed residuals and are usually far more effective. Preconditioning is the decisive step when the original coordinates make convergence slow.

The deeper unity is that every solver tries to expose directions in which the equation is easy. Elimination removes them one by one, stationary iteration repeatedly damps them, and Krylov methods build a subspace tailored to them. The matrix is the same; the algorithm decides how its geometry is revealed.